

Initiale Permutation der DES-Chiffre

Yusuf Enes Tamer

Juni 2025

1 Permutation auf Basis einer alternierenden Funktion

Es geht um den Permutation-Algorithmus des Data Encryption Standards. Mit der ist es möglich die Anordnung der vorgegeben 64 Bit-Werte zu vertauschen. Diese 64-Bit-Werte können in einer Matrix befinden, weshalb die Vertauschungen in einem 2D-Feld stattfinden. Ein Element im 2D-Feld wird durch die Zeilennummer sowie auch gleichzeitig von der Spaltennummer adressiert. Bei einer höheren Speicherkomplexität kann das 2D-Feld dupliziert werden, sodass die Bit-Werte in einer unterschiedlichen Anordnung in ein weiteres Feld geladen werden. Die Vielfachheit eines Feldes kann sich positiv auf die Laufzeit wirken, sodass die Permutation in einer schnelleren Zeitkomplexität stattfindet.

Das 2D-Feld wird folgendermaßen nummeriert:

5. 1. 6. 2. 7. 3. 8. 4.
 1.
 2.
 3.
 4.
 5.
 6.
 7.
 8.

Die Spaltennummer legt die Byte-Nummer fest, dagegen die Zeilennummer die Bit-Reihenfolge. Die unterschiedliche Anordnung der Byte-Nummern ist folgendermaßen festgelegt worden:

(8-3)	→	5	<—+ -4
(8-7)	→	1	<—+ +5
(8-2)	→	6	<—+ -4
(8-6)	→	2	<—+ +5
(8-1)	→	7	<—+ -4
(8-5)	→	3	<—+ +5
(8-0)	→	8	<—+ -4
(8-4)	→	4	<—+ -4

Die Permutation wird damit auf Basis einer alternierenden Addition/ Subtraktion Operation durchgeführt. Dieser Algorithmus kann in der Programmiersprache (C++) geschrieben werden:

```

for (int i = j; i < j+8; i++)
{
int q = 0;
for (int n = 1; n <= 4; n++)
    for (int l = n - 1; l <= n; l++)
        this->BitArrayV2[i][q++] = this->BitArray[5 * n - 4 * l - 1][i];
}

```

$$\sum_{n=1}^4 \sum_{l=n-1}^n 5n - 4l$$

Somit ist das Ziel die Permutation mindestens auf 2 Feldern durchzuführen, wo dabei die Zeilen und Spalten alterniert werden (Mirror). Diese Alternierung kann mit der vorliegenden Summen-Formel berechnet werden.

```

for (int i = j; i < j + 8; i++)
{
int q = 0;
for (int x = 0; x <= 1; x++)
for (int z = 1; z <= 4; z++)
{
if ( i <= 3)
this->BitArrayV3[i][q++] = this->BitArrayV2[8 - ( 2 * z - x )][7 - 1 - 2 * i];
else
this->BitArrayV3[i][q++] = this->BitArrayV2[8 - (2 * z - x)][7 - (i % 4) * 2];
}
}

```

Nachdem das Mirror-Feld erstellt wurde kann die einzelne Selektion von den einzelnen Bits beginnen. Dafür wird eine weitere Formel verwendet, die für die Adressierung dazu da ist:

$$\sum_{x=0}^1 \sum_{z=1}^4 (8 - (2z - x))$$

Der zweite Index kann folgendermaßen berechnet werden:

$$\sum_{i=0}^3 (7 - 2 * i - 1) \cup \sum_{i=4}^7 (7 - 2 * (i \bmod 4))$$

Dieser ist dazu da, um 8-Bitwerte zu extrahieren.

Die Inverse lässt sich folgendermaßen berechnen.

```

int q = -1;

for (int x = 1; x >= 0; x--)
{
    for (int i = 1; i <= 4; i++)
    {
        q++;
        for (int j = 0; j <= 7; j++)
        {
            if (x == 1)
            {
                this->BitArray[q][j] = this->BitArrayV2[
            }
            else
            {
                this->BitArray[q][j] = this->BitArrayV2[
            }
        }
    }
}

```

Für die Inverse-Permutation werden die folgenden Funktionen ausgeführt:

1. DES.mirrorV(0);
2. DES.selectInV(0);

1.	2.	3.	4.	5.	6.	7.	8.	
0	0	0	1	1	1	0	1	
9.	10.	11.	12.	13.	14.	15.	16.	
0	0	0	0	0	0	0	0	
17.	18.	19.	20.	21.	22.	23.	24.	
1	0	0	0	0	0	0	0	
25.	26.	27.	28.	29.	30.	31.	32.	
1	1	0	1	0	0	1	0	
33.	34.	35.	36.	37.	38.	39.	40.	
0	0	0	1	1	1	1	0	
41.	42.	43.	44.	45.	46.	47.	48.	
0	0	0	0	0	0	0	0	
49.	50.	51.	52.	53.	54.	55.	56.	
1	0	0	0	0	0	0	0	
57.	58.	59.	60.	61.	62.	63.	64.	
1	1	0	1	0	0	1	0	
58.	50.	42.	34.	26.	18.	10.	2.	= 0x88
1	0	0	0	1	0	0	0	
60.	52.	44.	36.	28.	20.	12.	4.	= 0x99
1	0	0	1	1	0	0	1	
62.	54.	46.	38.	30.	22.	14.	6.	= 0x11
0	0	0	1	0	0	0	1	
64.	56.	48.	40.	32.	24.	16.	8.	= 0x01
0	0	0	0	0	0	0	1	
57.	58.	50.	42.	34.	26.	18.	10.	= 0xcc
1	1	0	0	1	1	0	0	
59.	51.	43.	35.	27.	19.	11.	3.	= 0x00
0	0	0	0	0	0	0	0	
61.	53.	45.	37.	29.	21.	13.	5.	= 0x11
0	0	0	1	0	0	0	1	
63.	55.	47.	39.	31.	23.	15.	7.	= 0x98
1	0	0	1	1	0	0	0	